

C3 - Intelligence pour la robotique: TP7 – Apprentissage supervisé

Justin Ferdinand

I EXERCICES DE CLASSIFICATION

Dans cette partie, j'explore les résultats donnés par les expérimentations réalisées dans le notebook TP7_Classification.ipynb. Ce fichier contient en détail ces réalisations.

I.A CLASSIFICATION AVEC PERCEPTRON

Le Perceptron est un classifieur linéaire capable d'apprendre les fonctions logiques linéairement séparables. Les expériences suivantes démontrent cette capacité et ses limites.

I.A.1 QUESTION 1 - OPÉRATEUR OU

Le Perceptron a atteint une précision de 100% (accuracy = 1.0) pour l'opérateur OU, avec les prédictions : [0, 1, 1, 1], correspondant parfaitement aux valeurs attendues.

Cela vient du fait que l'opérateur OU est linéairement séparable dans le plan d'entrée. Une seule droite peut séparer les points de classe 0 (0,0) des points de classe 1 ((0,1), (1,0), (1,1)).

I.A.2 QUESTION 2 - OPÉRATEUR ET

Le Perceptron a également atteint une précision de 100% pour l'opérateur ET, avec les prédictions : [0, 0, 0, 1], correspondant aux valeurs attendues. L'opérateur ET est aussi linéairement séparable.

I.A.3 QUESTION 3 - OPÉRATEUR XOR

Le Perceptron n'a atteint qu'une précision de 50% pour l'opérateur XOR, avec les prédictions :

[0, 0, 0, 0], alors que les valeurs attendues sont [0, 1, 1, 0].

Cela vient du fait qu'il n'est pas possible de séparer les classes du XOR avec une seule ligne droite. Les points (0,1) et (1,0) sont de classe 1, tandis que (0,0) et (1,1) sont de classe 0, ce qui forme un motif en damier.

I.B MLP POUR XOR

Pour résoudre le problème non-linéaire du XOR, j'utilise un perceptron multicouche (MLP) avec une couche cachée.

I.B.1 QUESTIONS 4 & 5 - CONFIGURATION ET RÉSULTATS

Configuration du MLP :

- Activation : tanh
- Couches cachées : 1 couche de 2 neurones
- Max iterations : 10000
- Solver : lbfgs

Un premier entraînement a produit les prédictions : [0, 0, 1, 1], obtenant un score de 50%. Cependant, ce résultat correspond à une convergence partielle.

I.B.2 TEST DE STABILITÉ SUR 10 ESSAIS

Scores individuels : [1.0, 1.0, 0.75, 1.0, 1.0, 1.0, 0.5, 0.75, 0.75, 1.0] donc nombre de convergences réussies : 6 fois sur 10 (60%)

Le réseau peut converger vers une solution parfaite, mais l'entraînement est instable. L'initialisation aléatoire des poids ("random_state=None") entraîne une variabilité importante. Certains essais convergent complète-

ment (score = 1.0), tandis que d'autres se bloquent à des solutions limitées.

I.C EXTRACTION DES POIDS

I.C.1 QUESTION 6 - POIDS D'UN MODÈLE CONVERGENT

Pour un entraînement réussi (score = 1.0), les poids extraits du réseau MLP sont :

Couche cachée (Entrée → Cachée) :

- $w_1(x_1 \rightarrow h_1) = 3.4124$
- $w_2(x_1 \rightarrow h_2) = -3.5106$
- $w_3(x_2 \rightarrow h_1) = -3.9323$
- $w_4(x_2 \rightarrow h_2) = 3.5706$
- $b_1(\text{biais } h_1) = 2.4247$
- $b_2(\text{biais } h_2) = 1.8764$

Couche de sortie (Cachée → Sortie) :

- $w_5(h_1 \rightarrow \text{out}) = -8.6213$
- $w_6(h_2 \rightarrow \text{out}) = -8.3504$
- $b_3(\text{biais out}) = 8.7334$

I.C.2 ANALYSE DE LA DÉCISION

Les poids élevés en valeur absolue reflètent l'utilisation de la fonction d'activation tanh qui doit créer des frontières de décision nettes.

Pour le neurone h_1 , le poids positif important pour x_1 (3.4124) et le poids négatif pour x_2 (-3.9323) indiquent que h_1 s'active fortement lorsque x_1 est élevé et x_2 est faible, créant une partition du plan d'entrée. C'est le cas inverse pour le neurone h_2 .

Un biais de sortie fort et positif (8.7334) compense les poids négatifs de h_1 et h_2 pour créer la logique XOR.

I.D XOR EN RÉGRESSION

Pour résoudre XOR en utilisant une approche de régression plutôt que de classification :

Configuration :

- Modèle : MLPRegressor
- Architecture : 2 couches cachées de 4 neurones
- Activation : tanh

Résultats :

Prédictions (arrondies) : $[\approx 0.00, \approx 1.00, \approx 1.00, \approx 0.00]$

Score R^2 : 0.9999996073410653 (presque parfait)

Convergence en 126 itérations.

Je constate donc que l'approche par régression est plus stable et converge vers une solution quasi-parfaite. L'architecture plus profonde (deux couches cachées) facilite l'apprentissage de ce problème non-linéaire sans l'instabilité observée en classification.

II EXERCICE D'APPLICATION ROBOTIQUE

Dans cette partie, j'ai appliqué les concepts d'apprentissage supervisé à un problème de contrôle de robot en utilisant un réseau de neurones artificiels pour apprendre à éviter les obstacles.

II.A ENREGISTREMENT DES DONNÉES

J'ai utilisé le script `controleur_dataset_gen.py` pour enregistrer les données de capteurs et de commandes de vitesse du robot pendant une session de contrôle manuel. Le processus de collecte s'est déroulé comme suit :

Procédure de collecte :

- Contrôle manuel du robot Thymio via clavier dans le simulateur Webots
- Enregistrement synchrone des lectures capteurs et des commandes moteurs
- Plusieurs tours du circuit dans les deux sens pour diversifier les situations
- Sauvegarde au format HDF5 pour un accès efficace aux données

Données collectées :

- **Volume total :** 95 463 échantillons temporels
- **Capteurs de proximité :** 7 capteurs IR frontaux/latéraux (valeurs normalisées)
- **LiDAR :** scan laser 360° avec résolution angulaire fine

- **Caméra** : images RGB de l'environnement
- **Commandes moteurs** : vitesses gauche/droite appliquées (ground truth)

Format de stockage :

- Fichier : dataset_webots.hdf5
- Clés HDF5 : thymio_prox, thymio_scans, thymio_cam, thymio_commands
- Type : float32 pour optimiser mémoire et temps d'entraînement

Cette collecte par démonstration humaine constitue la base de l'apprentissage supervisé : le réseau apprendra à reproduire le comportement de pilotage observé dans ces données.

II.B PROTOCOLES DE TEST ET FUSION PROGRESSIVE DE CAPTEURS

II.B.1 ANALYSE COMPARATIVE DES FEATURES

Les modèles sont entraînés avec le notebook TP7_read_data.ipynb qui :

1. Charge le dataset HDF5 contenant les 95 463 échantillons sensoriels
2. Sélectionne les features selon le mode choisi
3. Applique une normalisation
4. Entraîne l'architecture MLP avec validation croisée
5. Sauvegarde le modèle si $R^2 > 0.94$

II.B.2 ANALYSE COMPARATIVE DES FEATURES

Pour mieux comprendre l'apport de chaque capteur, j'ai entraîné trois modèles distincts sur les mêmes données, en utilisant uniquement les capteurs de proximité, la caméra, ou le LiDAR.

Configuration commune :

- Architecture : 2 couches cachées de 64 neurones
- Activation : ReLU
- Solver : Adam (learning_rate = 1e-3)
- Early stopping : arrêt après 30 epochs sans amélioration
- Validation : 10% des données d'entraînement

Résultats obtenus :

Feature	R ² test	Loss finale	Itérations
Proximité	0.5708	0.035818	51
Caméra	0.8722	0.011327	448
LiDAR	0.9396	0.003695	108

II.B.3 INTERPRÉTATION DES COURBES D'APPRENTISSAGE

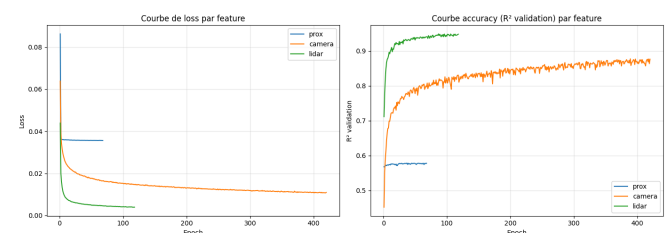


Figure 1: Courbes d'apprentissage pour les trois configurations de capteurs (Proximité, Caméra, LiDAR).

Les courbes de loss et d'accuracy (R^2 validation) visibles sur la Figure 1 révèlent des comportements d'apprentissage très différents selon le capteur utilisé :

Capteurs de proximité :

Le modèle converge très rapidement et atteint un plafond de performance trop limité pour un contrôle fiable ($R^2 \approx 0.57$). Les informations de proximités sont trop réduites pour permettre au

réseau d'apprendre un comportement efficace. Dans la plupart des frames, les capteurs de proximité ne détectent rien mais thymio engage des commandes moteurs, ce qui rend l'apprentissage difficile.

Caméra :

Le modèle avec la caméra est le plus long à converger avec 448 itérations. Sa courbe de loss décroît et montre que le modèle apprend plus longtemps. L'accuracy augmente lentement et plafonne à 0.87.

LiDAR :

Le LiDAR est un bon compromis car il converge rapidement tout en offrant une précision élevée par rapport aux autres capteurs.

Par la suite, j'ai testé la fusion de ces capteurs pour améliorer la précision du modèle.

II.C TEST 1 : CAPTEURS DE PROXIMITÉ UNIQUEMENT

La première expérience réalisée a consisté à utiliser le réseau exclusivement sur les données des capteurs de proximité pour tester que le modèle soit lisible et que les sorties moteurs soient compréhensibles par le robot.

Ce test n'a pas été concluant avec un score R^2 de 0.57, ce qui est insuffisant pour un contrôle fiable du robot. Cependant, cela m'a permis de comprendre comment exporter le modèle après son entraînement et de vérifier que les prédictions sont dans la bonne plage pour les commandes moteurs.

II.D TEST 2 : FUSION LiDAR + CAMÉRA

La deuxième expérience combine deux capteurs plus riches en information, le lidar et la caméra qui offrent les meilleures performances individuelles.

Ce modèle a obtenu un score R^2 de 0.92, ce qui est une amélioration significative par rapport au

test précédent. L'ajout du LiDAR et de la caméra a permis au réseau d'apprendre des représentations plus complexes de l'environnement, améliorant ainsi la précision des prédictions de vitesse. En test, Thymio était capable de suivre les murs mais avait parfois des difficultés dans les virages serrés où l'absence de capteur de proximité limitait la réactivité.

La capture vidéo de ce test s'intitule `lidarANDcamera_fail.mp4` et montre les limites de cette configuration, notamment dans les virages où le robot a tendance à couper les angles.

II.E TEST 3 : FUSION COMPLÈTE (LiDAR + CAMÉRA + PROXIMITÉ)

La dernière approche intègre tous les capteurs disponibles pour une perception maximale.

Pipeline de normalisation :

- 1 1. Conversion LiDAR 2D → normalisation [0,1]
- 2 2. Extraction caméra HSV → normalisation [0,1]
- 3 3. Normalisation proximité par L_1 (identique collecte de données)
- 4 4. StandardScaler sur ensemble d'entraînement
- 5 5. Prédiction vitesses moteurs [-1, 1]

Résultats d'évaluation :

- Score R^2 (Test) : 0.9654

Ce test avec la fusion de toutes les données a été le plus concluant, il a le meilleur score R^2 et Thymio est plus réactif, capable de suivre les murs et virages. J'observe qu'il fonctionne tout aussi bien dans un sens comme dans l'autre.

Cependant, j'observe une limite que j'avais déjà rencontrée lors d'un stage sur les bras robotiques intelligents par modèle VLA (Vision Language Action): Le robot est très peu capable de faire de la reprise à l'erreur.

Par exemple, en mettant le robot face à un mur, j'observe que son entraînement ne lui a pas permis d'apprendre à reculer ou à tourner sur place pour se dégager. Il continue à avancer et finit par heurter le mur, ce qui montre que le modèle n'a pas appris de stratégie de contournement ou de correction d'erreur. Il a simplement appris à sortir une commande moteur en fonction des données capteurs, mais sans une réelle compréhension de son environnement.

Pour améliorer cela, il faudrait enrichir le jeu de données par des scénarios d'échec pour que le modèle puisse apprendre à réagir face à ces situations. Il s'agissait là d'une des limites trouvée lors de mon stage où toutes les situations ne pouvaient pas être couvertes par les données d'entraînement, ce qui limitait la capacité du robot à faire face à des cas imprévus par l'ingénieur.

La capture vidéo de ce test s'intitule `lidarANDcameraANDprox.mp4` et montre une amélioration significative de la capacité du robot à suivre les murs et à négocier les virages.

II.F INTÉGRATION DU MODÈLE DANS WEBOTS

Procédure de chargement du modèle :

1. Initialisation Webots
2. Accès aux périphériques :
 - Moteurs gauche/droit (SetVelocity)
 - Caméra RGB
 - LiDAR
 - 7 capteurs de proximité normalisés
3. Chargement du modèle :
 - Path: Dans le contrôleur webots
 - Format: Pipeline (StandardScaler + MLPRegressor)
4. Boucle de contrôle :
 - Lecture capteurs bruts
 - Normalisation identique à l'entraînement
 - Passage au pipeline sklearn
 - Extraction prédictions [left_velocity, right_velocity]
 - Envoi commandes moteurs

Pour conclure, ce TP m'a permis de mettre en pratique les concepts d'apprentissage supervisé dans un contexte de contrôle robotique réel. Les expériences de classification (Perceptron, MLP) ont d'abord démontré les limites des classifieurs linéaires face à des problèmes non-linéaires comme le XOR, et la nécessité d'architectures multicouches pour approximer des fonctions complexes.

L'application au contrôle du robot Thymio a révélé l'importance cruciale du choix des capteurs. L'analyse comparative montre que le LiDAR surpasse nettement la caméra ($R^2 = 0.94$ vs 0.87) et les capteurs de proximité ($R^2 = 0.57$), grâce à une information structurée et peu bruitée. La fusion complète des trois modalités sensorielles atteint un score optimal de $R^2 = 0.9654$, permettant une navigation autonome fluide dans le circuit.

L'étude des courbes d'apprentissage a mis en évidence le rôle essentiel du mécanisme d'early stopping : il permet d'optimiser le temps d'entraînement en stoppant automatiquement lorsque le modèle atteint sa capacité maximale (51 epochs pour les capteurs de proximité limités en information) ou un plateau de performance (108 epochs pour le LiDAR). La caméra, plus complexe, nécessite 448 epochs pour converger.

Cependant, les tests en conditions réelles ont révélé une limite fondamentale de l'apprentissage supervisé par imitation : le robot ne peut gérer que les situations présentes dans les données d'entraînement. Face à un obstacle frontal, il ne sait pas reculer ou effectuer de manœuvre de dégagement, car ces comportements n'ont pas été démontrés pendant la collecte de données. Cette observation rejoint mon expérience en stage sur les modèles VLA (Vision-Language-Action) : la généralisation reste limitée sans une couverture exhaustive des cas d'usage.

Pour améliorer la robustesse du système, il serait nécessaire d'enrichir le dataset avec des scénarios de récupération d'erreur, ou d'explorer des approches d'apprentissage par renforcement qui permettraient au robot d'apprendre des stratégies de correction autonomes. L'apprentissage supervisé reste néanmoins une approche puissante pour le transfert de compétences humaines vers un système robotique, comme en témoigne le score $R^2 > 0.96$ atteint avec la fusion sensorielle complète.